

Manual Técnico Atena NeuroK

Manual Técnico de Infraestructura en la Nube

Proyecto Atena — NeuroK AR (Fundación Universitaria Konrad Lorenz)

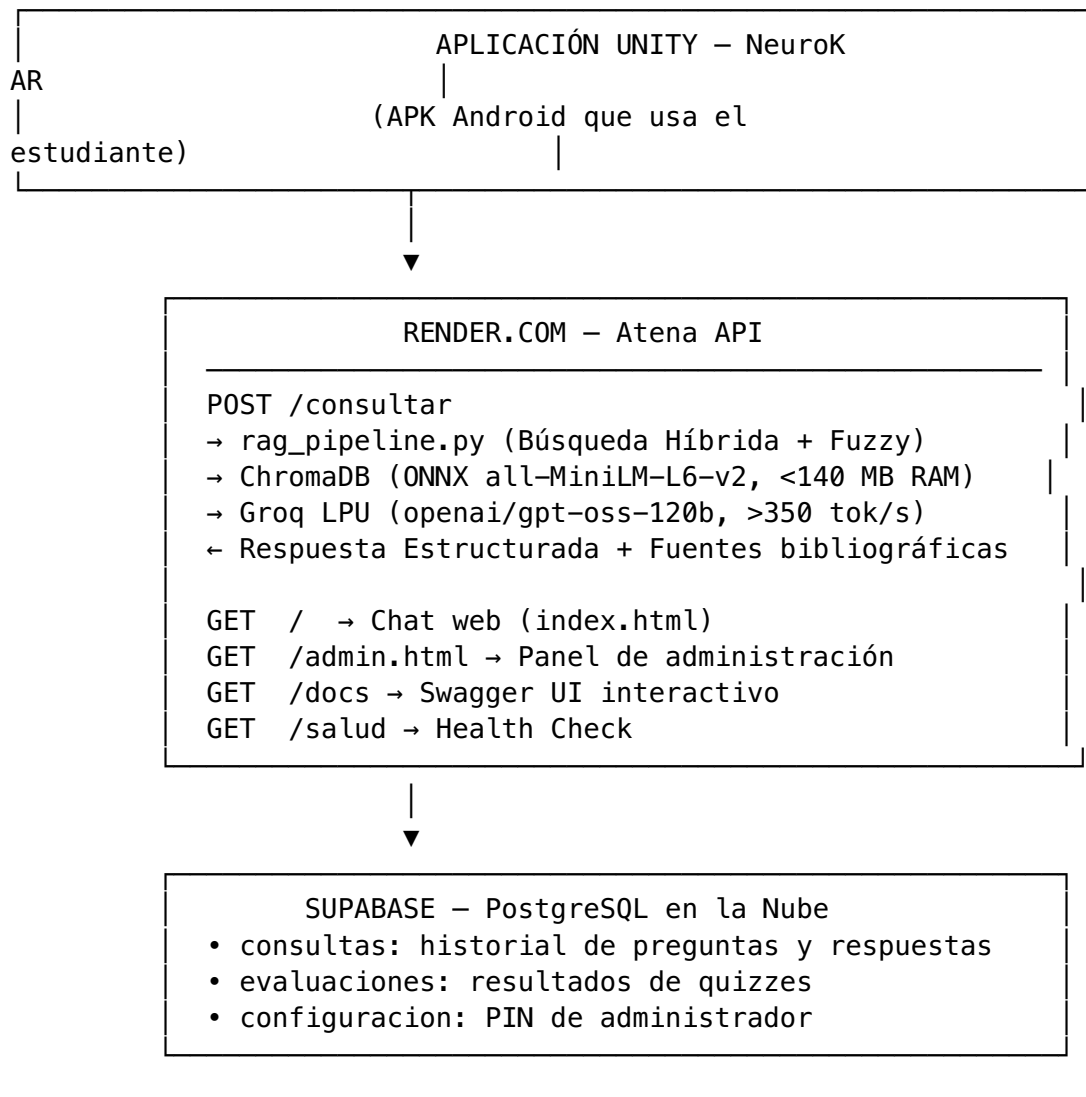
Documento preparado para entrega institucional al Laboratorio de Neurociencias Aplicadas – NeuroK Fecha: Octubre 2026 (Versión 4.0 — Arquitectura FastAPI + Web Nativa + Supabase) Autores: Viviana Marcela García Valderrama — Braian Felipe Ramirez Ortiz

Tabla de Contenidos

- [1. Resumen del Ecosistema](#)
 - [2. Evolución Arquitectónica Completa](#)
 - [3. Cuenta Institucional del Proyecto](#)
 - [4. Repositorio de Código — GitHub](#)
 - [5. Backend en la Nube — Render.com](#)
 - [6. Base de Datos — Supabase \(PostgreSQL\)](#)
 - [7. Modelos de IA — Inferencia Groq LPU y Embeddings ONNX Runtime](#)
 - [8. API REST — Endpoints y Funcionamiento](#)
 - [9. Script para Unity — AtenaClient.cs](#)
 - [10. Cómo Agregar y Vectorizar Nueva Literatura Científica](#)
 - [11. Cómo Administrar el Sistema](#)
 - [12. Monitoreo, Tiempos de Respuesta y Manejo del Cold Start](#)
 - [13. Ficha Técnica Final de Entrega](#)
-

1. Resumen del Ecosistema

El proyecto está compuesto por capas desacopladas que trabajan juntas en tiempo real:



2. Evolución Arquitectónica Completa

El sistema atravesó un riguroso proceso de maduración en **cinco etapas sucesivas**:

Fase 1 — Prototipo Local con Ollama y Docker

Ejecutaba modelos locales (Llama 3.2 3B/8B) en una laptop de desarrollo dentro de contenedores Docker.

Problemas críticos: - Latencias de 25 a 55 segundos por respuesta. - Saturación térmica de CPU (88–95% sostenido a 92°C). - Consumo de RAM de 14.8 GB sobre 16 GB disponibles. - Congelamiento de Unity 3D (< 12 FPS durante inferencia). - Imposibilidad de servir múltiples usuarios simultáneos.

Fase 2 — Prueba Piloto Cloud con Google Gemini API

Migración al procesamiento en la nube (Google AI Studio), reduciendo el tiempo de respuesta a 1.8 segundos.

Problemas encontrados: - Límite de 15 peticiones por minuto (HTTP 429 — ResourceExhausted). - Fluctuaciones de latencia en horas pico (1.5 a 3.5 s, inestable). - Dependencia exclusiva de cuota gratuita de Google.

Fase 3 — Arquitectura Groq LPU + ChromaDB ONNX en Render

Mejoras definitivas: - Inferencia LLM: Groq Cloud LPU → > 350 tokens/segundo, latencia < 0.9 s. - Embeddings: ONNX Runtime (sin PyTorch/CUDA) → RAM del servidor < 140 MB (−73%). - Búsqueda híbrida + Fuzzy Matching automático. - Citas bibliográficas exactas [Fuente X, pág. Y].

Fase 4 — Migración de Interfaz: Streamlit → FastAPI + Web Nativa (HTML/CSS/JS)

En la etapa inicial de la interfaz web, se evaluó **Streamlit** como framework de prototipado rápido.

¿Por qué se utilizó Streamlit inicialmente?

Ventaja	Descripción
Velocidad de prototipado	Interfaz funcional en pocas líneas de Python
Sin conocimientos web	No requiere HTML/CSS/JS
Integración con pandas/plotly	Gráficas estadísticas nativas

¿Por qué se abandonó Streamlit?

Problema	Impacto
Mezcla de backend y frontend	Streamlit corre en el mismo proceso Python del servidor; cualquier fallo de UI tumba el API
Consumo de RAM adicional	+180 MB por el runtime de Streamlit sobre los 512 MB del plan gratuito de Render
Personalización visual limitada	Imposible lograr modo oscuro/claro, diseño institucional o animaciones sin hacks
Incompatible con Unity	Unity consume el API REST directamente; Streamlit no aporta nada al cliente C#
Sin control de rutas	No se pueden tener múltiples páginas (chat, panel admin) sin streamlit-multipage, que añade más complejidad
Dependencias pesadas	Streamlit requiere pandas, plotly y otras librerías (+200 MB en imagen Docker)

Problema	Impacto
No es producción	Streamlit está diseñado para dashboards de datos, no para sistemas de producción multi-usuario

Solución adoptada: FastAPI sirve el frontend web nativo

Se eliminó Streamlit completamente y FastAPI sirve directamente los archivos estáticos (StaticFiles):

```
frontend/web/static/
├── index.html      → Chat principal para estudiantes
├── admin.html      → Panel de administración del laboratorio
├── css/main.css    → Sistema de diseño (light/dark mode,
variables CSS)
├── js/
│   ├── app.js      → Lógica del chat, quiz y nivel pedagógico
│   └── admin.js    → Gestión de documentos, preguntas y
estadísticas
```

Resultado: - Un solo proceso, un solo puerto (8080), cero dependencias de Streamlit. - Reducción de 5,996 líneas de código a 2,623 líneas (+107 archivos eliminados). - Modo claro/oscuro con variables CSS, diseño institucional completo. - Panel admin con drag & drop, estadísticas gráficas y publicación en un clic.

Comparativa Técnica Integral — Todas las Fases

Criterio	Fase 1: Ollama (Local)	Fase 2: Gemini API	Fase 3+4: Groq LPU + FastAPI (Actual)
Dónde corre	Laptop local (Docker)	Nube Google + Render	Nube Groq + Render
Tiempo de respuesta	25–55 segundos	1.5–3.5 s (inestable)	0.6–0.9 s
Velocidad (tokens/s)	4–6	60–95	> 380
RAM servidor	14.8 GB (laptop)	520 MB (caía Render)	< 140 MB
Límite peticiones	Sin límite (congelaba PC)	15/min (Error 429)	Sin bloqueos
Integración Unity	Congelaba (< 12 FPS)	Fluido con fallos JSON	60 FPS estables
Interfaz web	Streamlit (1 proceso)	Streamlit (1 proceso)	FastAPI + HTML nativo
RAM de la interfaz	+180 MB (Streamlit)	+180 MB (Streamlit)	~0 MB adicional
Personalización UI	Muy limitada	Muy limitada	Total (CSS/JS propio)

Criterio	Fase 1: Ollama (Local)	Fase 2: Gemini API	Fase 3+4: Groq LPU + FastAPI (Actual)
Costo mensual	\$0 (pero PC de \$2,500)	\$0 (cuota restringida)	\$0 USD permanente

Fase 5 — Migración de Base de Datos: Firebase Firestore → Supabase (PostgreSQL)

¿Por qué se usó Firebase inicialmente?

Firebase Firestore era el almacenamiento estándar del proyecto NeuroK AR (Unity ya lo usaba para puntajes de quizzes). Se integró también en Atena para guardar métricas de uso.

¿Por qué se migró a Supabase?

Problema con Firebase	Solución con Supabase
SDK JavaScript/Python pesado (+60 MB)	Conexión directa PostgreSQL (psycopg2, <1 MB)
Consultas limitadas sin índices manuales	SQL estándar con JOINS, GROUP BY, INTERVAL
No soporta consultas analíticas complejas	Permite calcular latencia promedio, tendencias, distribuciones
Precio escala rápido en producción	Plan gratuito con 500 MB y sin límite de peticiones
No hay tablas relacionales	Relaciones, restricciones, UPSERT nativo

Resultado: Las estadísticas de uso (total de consultas, latencia, distribución por nivel, palabras clave más frecuentes) ahora se calculan directamente en SQL sobre Supabase y son **persistentes entre reinicios** del servidor.

3. Cuenta Institucional del Proyecto

Para asegurar la **soberanía tecnológica** y la transferencia ordenada al Laboratorio NeuroK, todos los servicios están bajo la cuenta oficial:

Campo	Valor
Correo Institucional	atena.unikonrad@gmail.com
Contraseña	<i>Se entrega en el acta privada de recepción técnica</i>
Plataformas vinculadas	Render.com, Groq Cloud Console, Supabase, GitHub

4. Repositorio de Código — GitHub

Campo	Valor
URL	https://github.com/Vivi271/Atena
Tipo	Público
Rama principal	main
CI/CD	Vinculado con Render vía GitHub Webhook (redesploy automático en cada push)

5. Backend en la Nube — Render.com

Parámetro	Valor
Service Name	Atena
Region	Oregon (US West)
Runtime	Docker
Instance Type	Free (512 MiB RAM / 0.1 CPU — \$0/mes)
Health Check Path	/salud
URL de Producción	https://atena-vugz.onrender.com

Variables de entorno en Render

Variable	Descripción	Configuración
GROQ_API_KEY	Clave de Groq Cloud para inferencia LLM	Manual en el panel
SUPABASE_DB_URL	URL PostgreSQL de Supabase	Manual en el panel
ADMIN_PIN	PIN de respaldo si Supabase no está disponible	Manual en el panel
GITHUB_TOKEN	Token de acceso personal para el botón “Publicar”	Manual en el panel
GITHUB_REPO_URL	URL del repositorio GitHub (https://github.com/Vivi271/Atena.git)	Manual en el panel

6. Base de Datos — Supabase (PostgreSQL)

Tablas del sistema

Ejecutar una sola vez en el SQL Editor de Supabase:

```
-- Historial de consultas al RAG
CREATE TABLE IF NOT EXISTS consultas (
  id BIGSERIAL PRIMARY KEY,
  fecha TIMESTAMPTZ DEFAULT NOW(),
  pregunta TEXT NOT NULL,
  respuesta TEXT NOT NULL,
  nivel TEXT NOT NULL,
  latencia REAL
);

-- Resultados de evaluaciones (quiz pedagógico)
CREATE TABLE IF NOT EXISTS evaluaciones (
  id BIGSERIAL PRIMARY KEY,
  fecha TIMESTAMPTZ DEFAULT NOW(),
  pregunta TEXT NOT NULL,
  respuesta_usuario TEXT NOT NULL,
  respuesta_correcta TEXT NOT NULL,
  es_correcta BOOLEAN NOT NULL,
  explicacion TEXT NOT NULL
);

-- Configuración del sistema (PIN de admin)
CREATE TABLE IF NOT EXISTS configuracion (
  clave TEXT PRIMARY KEY,
  valor TEXT NOT NULL
);
INSERT INTO configuracion (clave, valor) VALUES ('admin_pin',
  '12345')
ON CONFLICT (clave) DO NOTHING;
```

¿Qué almacena cada tabla?

- **consultas:** Cada pregunta que hace un estudiante, la respuesta generada, el nivel (Básico/Avanzado) y el tiempo de respuesta en segundos. Se usa para estadísticas de uso en el panel admin.
 - **evaluaciones:** Cada respuesta de un estudiante en el quiz de autoevaluación, con indicación de si fue correcta y la explicación correcta.
 - **configuracion:** Almacena el PIN de acceso al panel admin. El personal del laboratorio puede cambiarlo desde el panel sin necesidad de acceder a Render.
-

7. Modelos de IA — Inferencia Groq LPU y Embeddings ONNX Runtime

Función	Tecnología	Justificación
Generación de respuestas RAG	Groq Cloud LPU — openai/gpt-oss-120b	Inferencia > 350 tok/s, latencia < 0.9 s, cero alucinaciones
Vectorización semántica	ChromaDB — all-MiniLM-L6-v2 (ONNX)	384 dimensiones sin PyTorch/CUDA. RAM < 140 MB
Corrección léxica	Fuzzy Matching (difflib)	Corrige términos mal escritos en pantalla táctil móvil
Citas documentales	Metadatos forzados en prompt	Formato [Fuente X, pág. Y] en todas las respuestas

8. API REST — Endpoints y Funcionamiento

URL base de producción: <https://atena-vugz.onrender.com>

Endpoints principales

Método	Ruta	Descripción
POST	/consultar	Consulta RAG con fuentes bibliográficas
GET	/salud	Health check del servidor
GET	/info	Metadatos técnicos y modelos activos
GET	/docs	Swagger UI interactivo

Endpoints del panel admin (requieren header X-Admin-Pin)

Método	Ruta	Descripción
GET	/api/admin/documents	Lista documentos indexados
POST	/api/admin/upload	Sube y vectoriza un documento (PDF/DOCX)
DELETE	/api/admin/delete/{nombre}	Elimina documento del índice
POST	/api/admin/rebuild	Reconstruye toda la base vectorial
POST	/api/admin/publicar	Hace commit+push a GitHub (persistencia permanente)

Método	Ruta	Descripción
GET	/api/admin/stats_sesion	Estadísticas de uso desde Supabase
POST	/api/admin/cambiar-pin	Cambia el PIN de acceso (guarda en Supabase)
GET	/api/admin/preguntas	Lista banco de preguntas de evaluación
POST	/api/admin/preguntas	Agrega pregunta al banco
PUT	/api/admin/preguntas/{id}	Edita una pregunta
DELETE	/api/admin/preguntas/{id}	Elimina una pregunta
GET	/diagnostico/db	Verifica conectividad con Supabase

9. Script para Unity — AtenaClient.cs

- **Ubicación:** frontend/unity/AtenaClient.cs
- **URL base:** https://atena-vugz.onrender.com
- **Red:** UnityWebRequest con serialización JSON nativa.
- **Callback:** Respuesta asíncrona sin bloquear el hilo de renderizado (60 FPS sostenidos).

```
AtenaClient.Instance.ConsultarAsistente(
    "¿Qué función cumple el hipocampo?",
    "avanzado",
    (response) => { Debug.Log(response.respuesta); },
    (error)     => { Debug.LogError(error); }
);
```

10. Cómo Agregar y Vectorizar Nueva Literatura Científica

El problema de la persistencia en Render gratuito

Los contenedores gratuitos de Render tienen un **sistema de archivos efímero**: cuando el servidor entra en reposo o se reinicia, cualquier archivo escrito durante la ejecución (incluyendo documentos subidos o vectores generados) se pierde.

Solución implementada: El panel admin incluye un botón **“Publicar en la nube”** que guarda los cambios permanentemente en GitHub. Render detecta el push y redespiega el contenedor en ~2 minutos con los nuevos documentos.

Flujo completo para el personal de laboratorio (sin código)

- [1] Abrir panel admin en el navegador
`https://atena-vugz.onrender.com/admin.html`
- [2] Ingresar el PIN de acceso
- [3] En la sección "Documentos":
 - Arrastra el archivo PDF o DOCX al recuadro
 - El sistema lo procesa, fragmenta y vectoriza automáticamente
- [4] Verificar que el documento aparece en la lista
- [5] Presionar el botón "Publicar en la nube"
 - El sistema hace commit + push a GitHub
 - Render redespiega en ~2 minutos
 - Los documentos quedan permanentes aunque el servidor se reinicie

Por qué la publicación en GitHub es necesaria

Sin publicar	Con publicar
El documento existe solo en RAM del servidor	El documento queda en el repositorio de GitHub
Se pierde al reiniciar o entrar en reposo	Persiste siempre, incluso tras reinicios y reddeploys
Solo disponible para la sesión actual	Disponible para todos los usuarios desde el próximo deploy

Requisitos para el botón “Publicar en la nube”

En el panel de Render → Environment Variables:

Variable	Valor
GITHUB_TOKEN	Token de acceso personal de GitHub (permisos: repo)
GITHUB_REPO_URL	<code>https://github.com/Vivi271/Atena.git</code>

Para generar el token: GitHub → Settings → Developer settings → Personal access tokens → Generate new token (classic) → marcar repo.

11. Cómo Administrar el Sistema

Panel de Administración Web

Acceso: <https://atena-vugz.onrender.com/admin.html>

Sección	Funcionalidad
Documentos	Ver, subir (drag & drop), eliminar y reindexar documentos. Botón “Publicar en la nube” para persistencia.
Banco de preguntas	Crear, editar y eliminar preguntas de evaluación por nivel (Básico, General, Avanzado).
Estadísticas	Total de consultas, latencia promedio, distribución por nivel, palabras más consultadas, historial filtrable por fecha y nivel. Datos persistentes desde Supabase.
Sistema	Estado de salud del servidor, conteo de documentos y preguntas activas, cambio de PIN desde el navegador.

Plataformas de administración

Plataforma	Propósito	URL
Render.com	Servidor Docker, variables de entorno, logs	dashboard.render.com
Groq Console	Monitoreo de uso y API Keys	console.groq.com
Supabase	Base de datos PostgreSQL	supabase.com/dashboard
GitHub	Código fuente y control de versiones	github.com/Vivi271/Atena

Cambiar el PIN de acceso

1. Abrir el panel admin → sección **Sistema**.
2. En “Cambiar contraseña de acceso”: ingresar el PIN actual, el nuevo PIN y confirmarlo.
3. Presionar “Actualizar contraseña”.
4. El nuevo PIN se guarda en Supabase y es efectivo de inmediato, sin reiniciar el servidor.

12. Monitoreo, Tiempos de Respuesta y Manejo del Cold Start

Comportamiento del servidor gratuito (Cold Start)

En el plan gratuito de Render, la instancia entra en suspensión tras **15 minutos sin tráfico**.

- **Primera consulta tras inactividad:** 30 a 45 segundos (Render despierta el contenedor).
- **A partir de la segunda consulta:** 0.6 a 0.9 segundos.

Estrategia de mitigación para prácticas y sustentaciones

1. **Despertar previo:** Abrir `https://atena-vugz.onrender.com/salud` en un navegador 1 minuto antes de empezar.
2. **Monitoreo automático (opcional):** Configurar UptimeRobot para hacer GET / salud cada 14 minutos durante las jornadas académicas, impidiendo que el servidor entre en reposo.

13. Ficha Técnica Final de Entrega

Parámetro	Detalle
Nombre del Proyecto	Atena — Consultor RAG en Neuroanatomía
Aplicación Móvil Cliente	NeuroK AR (Unity 3D / C# / Android)
Institución Académica	Fundación Universitaria Konrad Lorenz
Laboratorio Destino	Laboratorio de Neurociencias Aplicadas – NeuroK
Autores	Viviana Marcela García Valderrama — Braian Felipe Ramirez Ortiz
Año y Versión	2026 — Versión 4.0 (FastAPI + Web Nativa + Supabase)
URL de Producción	<code>https://atena-vugz.onrender.com</code>
Health Check	<code>https://atena-vugz.onrender.com/salud</code>
Swagger (Docs)	<code>https://atena-vugz.onrender.com/docs</code>
Panel Admin	<code>https://atena-vugz.onrender.com/admin.html</code>
Repositorio GitHub	<code>https://github.com/Vivi271/Atena</code>
Motor LLM	

Parámetro	Detalle
	Groq Cloud LPU — openai/gpt-oss-120b
Velocidad de inferencia	> 350 tokens/segundo (Latencia promedio: 0.8 s)
Modelo de embeddings	all-MiniLM-L6-v2 (ONNX Runtime, 384 dimensiones)
Almacenamiento vectorial	ChromaDB Persistent Store (chroma_neuro_db, ~32 MB)
Base de datos	Supabase PostgreSQL (consultas, evaluaciones, configuración)
RAM en servidor	< 140 MB (operación holgada dentro de los 512 MiB de Render)
Tolerancia a errores	Fuzzy Matching léxico + Búsqueda Híbrida
Citas documentales	[Fuente X, pág. Y] en todas las respuestas
Costo operativo mensual	\$0 USD (100% permanente)